

深圳大学实验报告

课程名称： 编译原理

实验项目名称： 自顶向下的语法分析程序设计

学院： 计算机与软件学院

专业： 软件工程

指导教师： 王祎乐

报告人： 学号： 班级：

实验时间： 2026年5月8日至5月24日

实验报告提交时间： 2026年5月11日

教务处制

实验目的与要求:

目的: 通过亲自设计、编码并调试一个 **LL(1) 预测分析器**, 深化对上下文无关文法、FIRST / FOLLOW 集、LL(1) 分析表与预测分析算法的理解; 掌握将理论算法转化为可运行程序的能力, 并学会用表格方式直观展示语法分析过程。

要求:

1. 正确计算 **FIRST**、**FOLLOW** 集并打印结果。
2. 按照**算法构造 LL(1) 分析表**
3. 对任意输入串执行 LL(1) 预测分析, 实时输出“步骤 / 分析栈 / 输入串 / 所用产生式”表格, 遇到错误能定位并报告。

推荐测试的表达式:

$i+i*i$, $i*(i+i)$ 以及错误表达式 $i+)i$, 例如: $1+2*3$, $2*(3+4)$, $1+)2$, 可以多试几个
更多补充内容看 [README.md/pdf](#)

方法、步骤:

要完成本实验, 依据实验要求进行分解, 需要完成的实验步骤是:

1. FIRST 集的求解过程是?

- (1) **初始化:** 对终结符 a , $FIRST(a) = \{a\}$; 对非终结符 A , $FIRST(A) = \emptyset$ 。
- (2) **迭代处理产生式:** 对每个产生式 $A \rightarrow a$, 按 a 中符号顺序处理:
 - 若 a 以终结符 a 开头, 将 a 加入 $FIRST(A)$ 。
 - 若 a 以非终结符 B 开头, 将 $FIRST(B)$ 加入 $FIRST(A)$ 。若 $\epsilon \in FIRST(B)$, 继续处理 a 后续符号 (重复此过程)。
- (3) **终止条件:** 反复迭代, 直到所有非终结符的 FIRST 集不再变化。

2. FOLLOW 集的求解过程是?

- (1) **初始化:** 对开始符号 S , $FOLLOW(S) = \{\#\}$ ($\#$ 为输入串结束标记); 其他非终结符 FOLLOW 集初始化为 $\emptyset$ 。
- (2) **迭代处理产生式:** 对每个产生式 $A \rightarrow aB\beta$, 按 a 中符号顺序处理:
 - 求 $FIRST(\beta)$, 将其中非 ϵ 元素加入 $FOLLOW(B)$
 - 若 $\epsilon \in FIRST(\beta)$, 将 $FIRST(A)$ 加入 $FOLLOW(B)$ 。
- (3) **终止条件:** 反复迭代, 直到所有非终结符的 FIRST 集不再变化。

3. 已知 FIRST 集和 FOLLOW 集如何得到 LL(1) 分析表?

- (1) **迭代处理产生式:** 对每个产生式 $A \rightarrow a$
 - 计算 $FIRST(a)$, 若 $a \in FIRST(a)$ (a 为终结符), 则 $M[A, a] = A \rightarrow a$
 - 若 $\epsilon \in FIRST(a)$, 对 $FOLLOW(A)$ 中每个终结符或 $\#$ (记为 b), 令 $M[A, b] = A \rightarrow a$
- (2) **终止条件:** 其余无定义的表项 $M[A, a]$ 标记为 “错误”。

4. 根据 LL(1) 分析表进行 LL(1) 预测分析的步骤是?

- (1) **初始化:** 将 $\#$ 和开始符号 S 压入栈, 输入指针指向输入串首字符
- (2) **循环分析:**
 - 取出栈顶符号 X

- 若 X 是终结符：

若 X 与当前输入符号匹配，输入指针后移，弹出 X 。

否则，报错（输入串不合法）。

- 若 X 是非终结符：

查分析表 $M[X, a]$ (a 为当前输入符号)。

若 $M[X, a]$ 有产生式 $X \rightarrow \beta$ ，弹出 X ，将 β 逆序压栈 (β 为 ϵ 时不压栈)。

若 $M[X, a]$ 无定义，报错（输入串不合法）。

(3) 终止条件：栈空且输入指针指向 #，分析成功，输入串合法

实验过程及内容：

1. First 集求解

设计说明：

根据编译原理知识，FIRST 集的计算需要对每个非终结符进行迭代，直到集合不再变化。对于产生式右部，如果是终结符，直接加入 FIRST 集；如果是非终结符，递归加入其 FIRST 集，遇到空产生式则继续处理后面的符号。在代码中，需要遍历每个非终结符的所有产生式，对每个符号进行处理。注意，当处理到空字符串（比如 "ε"）时，需要将空加入 FIRST 集。

编程过程：

(1) 开始使用 while 循环进行迭代

(2) 然后发现问题：FIRST 集为空

(3) 分析问题原因：FIRST 集的计算需要从初始状态开始逐步迭代，直到所有集合收敛（不再变化）。即使初始时所有 FIRST 集为空，也需要通过迭代填充它们。例如：

- 若有产生式 $A \rightarrow B$ 且 $B \rightarrow a$ ，则 $FIRST(A)$ 最终应为 a 。但这需要先计算 $FIRST(B)$ ，再将其传递给 $FIRST(A)$ 。
- 若使用普通 while 循环且初始条件不满足，上述传递过程将被跳过，导致 $FIRST(A)$ 仍为空。

(4) 最后使用 do-while 循环进行迭代

修改代码如下：

```
46 // 计算FIRST集
47 void Sol_First() {
48     // 初始化：终结符的FIRST集是自身
49     for (auto &vt : Vt) {
50         if (vt != "ε")
51             FIRST[vt] = {vt};
52     }
53     // 初始化非终结符FIRST集为空
54     for (auto &vn : Vn) {
55         FIRST[vn].clear();
56     }
57
58     bool updated;
59     // 迭代计算直到不再变化
60     do {
61         updated = false;
62         for (auto &vn : Vn) {
63             for (auto &prod : pro[vn]) {
64                 set<string> cur_first;
65                 bool can_empty = true;
66
67                 // 遍历产生式右部每个符号
68                 for (int i = 0; i < prod.size(); ++i) {
69                     string s(1, prod[i]);
70                     // 遇到终结符，直接加入并终止
71                     if (Vt.count(s)) {
72                         cur_first.insert(s);
73                         can_empty = false;
74                         break;
75                     }
76                 }
77                 if (can_empty) cur_first.insert("");
78                 if (cur_first != FIRST[vn]) {
79                     FIRST[vn] = cur_first;
80                     updated = true;
81                 }
82             }
83         }
84     } while (updated);
85 }
```

```

76 // 遇到非终结符，加入其FIRST集
77 if (Vn.count(s)) {
78     cur_first.insert(FIRST[s].begin(), FIRST[s].end());
79     // 不能推出空，终止
80     if (!FIRST[s].count("ε")) {
81         can_empty = false;
82         break;
83     }
84 }
85 }
86
87 // 全部符号都能推出空，加入ε
88 if (can_empty || prod.empty()) {
89     cur_first.insert("ε");
90 }
91
92 // 更新FIRST集
93 for (auto &x : cur_first) {
94     if (!FIRST[vn].count(x)) {
95         FIRST[vn].insert(x);
96         updated = true;
97     }
98 }
99 }
100 }
101 } while (updated);

102
103 // 输出FIRST集
104 cout << "\nFIRST集:" << endl;
105 for (auto &p : FIRST) {
106     if (Vn.count(p.first)) {
107         cout << "FIRST(" << p.first << ")={";
108         bool first = true;
109         for (auto &x : p.second) {
110             if (!first) cout << ",";
111             cout << x;
112             first = false;
113         }
114         cout << "}" << endl;
115     }
116 }
117 }

```

2. Follow 集求解

设计说明：

FOLLOW 集的计算首先将开始符号的 FOLLOW 集初始化为包含 #，然后对于每个产生式 $A \rightarrow \alpha B \beta$ ，将 $FIRST(\beta)$ 中除空以外的元素加入 $FOLLOW(B)$ ，如果 β 可以推导出空，则将 $FOLLOW(A)$ 加入 $FOLLOW(B)$ 。对于 $A \rightarrow \alpha B$ 或 $A \rightarrow \alpha B \beta$ 且 β 推导出空的情况，都需要处理。代码中需要迭代处理，直到 FOLLOW 集不再变化，可能需要使用一个循环，每次检查是否有新元素加入。

编程过程：

- (1) 开始符号的 FOLLOW 集未正确初始化为 {#}，而是全部清空。
- (2) 然后发现问题：FOLLOW 集输出不正确
- (3) 分析问题原因：FOLLOW 集的定义与分析过程的终止条件强相关。分析过程以栈初始化为 #S 开始（# 是栈底结束符，S 是开始符号）。当 S 推导完毕后，栈顶会回到 #，此时输入串也应处理到 #，才能成功结束分析。若 FOLLOW(S) 未包含 #，则无法正确识别输入串的结束条件。
- (4) 修改代码如下：

```

119 // 计算FOLLOW集
解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
120 void Sol_Follow() {
121     // 初始化所有非终结符FOLLOW集为空
122     for (auto &vn : Vn)
123         FOLLOW[vn].clear();
124     // 开始符号FOLLOW加入#
125     FOLLOW[string(1, start)].insert("#");
126
127     bool updated;
128     // 迭代计算直到不再变化
129     do {
130         updated = false;
131         for (auto &A : Vn) {
132             for (auto &alpha_str : pro[A]) {
133                 int len = alpha_str.size();
134                 // 遍历产生式右部每个符号B
135                 for (int i = 0; i < len; ++i) {
136                     string B(1, alpha_str[i]);
137                     if (!Vn.count(B)) continue;
138
139                     string beta = alpha_str.substr(i + 1);
140                     set<string> first_beta;
141                     bool beta_empty = true;
142
143                     // 计算FIRST(beta)
144                     for (int j = 0; j < beta.size(); ++j) {
145                         string s(1, beta[j]);
146                         if (Vt.count(s)) {
147                             first_beta.insert(s);
148                             beta_empty = false;
149                             break;
150                         }
151                         if (Vn.count(s)) {
152                             first_beta.insert(FIRST[s].begin(), FIRST[s].end());
153                             if (!FIRST[s].count("ε")) {
154                                 beta_empty = false;
155                                 break;
156                             }
157                         }
158                     }
159                     if (beta.empty() || beta_empty)
160                         first_beta.insert("ε");
161
162                     // 将FIRST(beta)-{ε}加入FOLLOW(B)
163                     set<string> to_add;
164                     for (auto &x : first_beta) {
165                         if (x != "ε")
166                             to_add.insert(x);
167                     }
168                     // 若beta可空, 将FOLLOW(A)加入FOLLOW(B)
169                     if (first_beta.count("ε")) {
170                         to_add.insert(FOLLOW[A].begin(), FOLLOW[A].end());
171                     }
172
173                     // 更新FOLLOW集
174                     for (auto &x : to_add) {
175                         if (!FOLLOW[B].count(x)) {
176                             FOLLOW[B].insert(x);
177                             updated = true;
178                         }
179                     }
180                 }
181             }
182         }
183     } while (updated);
184
185     // 输出FOLLOW集
186     cout << "\nFOLLOW集:" << endl;
187     for (auto &p : FOLLOW) {
188         cout << "FOLLOW(" << p.first << ")={";
189         bool first = true;
190         for (auto &x : p.second) {
191             if (!first) cout << ",";
192             cout << x;
193             first = false;
194         }
195         cout << "}" << endl;
196     }
197 }

```

3. 构建 LL(1)分析表

设计说明:

构造分析表 M 。对于每个产生式 $A \rightarrow \alpha$ ，计算 $FIRST(\alpha)$ ，对于每个 a 属于 $FIRST(\alpha)$ 且 a 不为空，将 $M[A, a]$ 设为该产生式。如果 α 可以推导出空，则对于每个 b 属于 $FOLLOW(A)$ ，将 $M[A, b]$ 设为该产生式。需要注意处理空字符串的情况，比如产生式为 "e" 时，代表空产生式。

代码:

```
199 // 构造LL(1)分析表
    解释代码 | 生成文档 | 修复代码 | 生成测试 | 代码评审 | 关闭
200 void Build_LL1() {
201     M.clear();
202     for (auto &A : Vn) {
203         for (auto &prod : pro[A]) {
204             set<string> first_p;
205             bool p_empty = true;
206
207             // 计算当前产生式右部的FIRST集
208             for (int i = 0; i < prod.size(); ++i) {
209                 string s(1, prod[i]);
210                 if (Vt.count(s)) {
211                     first_p.insert(s);
212                     p_empty = false;
213                     break;
214                 }
215                 if (Vn.count(s)) {
216                     first_p.insert(FIRST[s].begin(), FIRST[s].end());
217                     if (!FIRST[s].count("e")) {
218                         p_empty = false;
219                         break;
220                     }
221                 }
222             }
223             if (prod.empty() || p_empty)
224                 first_p.insert("e");
225
226             // 填充分析表
227             for (auto &a : first_p) {
228                 if (a == "e") {
229                     // 可空, 用FOLLOW(A)填充
230                     for (auto &b : FOLLOW[A]) {
231                         if (b == "#" || Vt.count(b)) {
232                             M[{A[0], b[0]}] = prod;
233                         }
234                     }
235                 } else {
236                     // 不可空, 直接用FILL
237                     if (Vt.count(a) || a == "#") {
238                         M[{A[0], a[0]}] = prod;
239                     }
240                 }
241             }
242         }
243     }
244 }
```

4. 验证准确性

(1) 样例测试, grammar.txt, 输入串 1+2*3, 运行结果如下:

```

● → /workspace git:(master) X g++ -std=c++11 LL1Parser.cpp -o ll1_parser
● → /workspace git:(master) X ./ll1_parser
请输入文法文件名（例如 grammar.txt）： examples/grammar.txt
请输入要分析的输入串： 1+2*3

```

非终结符(V_n): A B E F T

终结符(V_t): () * + e i

存在左递归，消除后如下：

A $\rightarrow$ +TA

A $\rightarrow$ e

B $\rightarrow$ *FB

B $\rightarrow$ e

E $\rightarrow$ TA

F $\rightarrow$ (E)

F $\rightarrow$ i

T $\rightarrow$ FB

FIRST集：

FIRST(A)={+,e}

FIRST(B)={*,e}

FIRST(E)={(,i}

FIRST(F)={(,i}

FIRST(T)={(,i}

FOLLOW集：

FOLLOW(A)={#,)}

FOLLOW(B)={#,),+}

FOLLOW(E)={#,)}

FOLLOW(F)={#,),*,+}

FOLLOW(T)={#,),+}

LL(1) 分析表 (表格形式):

	#	(	)	*	+	e	i
A	A $\rightarrow$ e		A $\rightarrow$ e		A $\rightarrow$ +TA		
B	B $\rightarrow$ e		B $\rightarrow$ e	B $\rightarrow$ *FB	B $\rightarrow$ e		
E		E $\rightarrow$ TA					E $\rightarrow$ TA
F		F $\rightarrow$ (E)					F $\rightarrow$ i
T		T $\rightarrow$ FB					T $\rightarrow$ FB

转换后输入串为: i+i*i

分析过程:

步骤	分析栈	输入串	所用产生式
1	#E	i+i*i#	E->TA
2	#AT	i+i*i#	T->FB
3	#ABF	i+i*i#	F->i
4	#ABi	i+i*i#	
5	#AB	+i*i#	B->e
6	#A	+i*i#	A->+TA
7	#AT+	+i*i#	
8	#AT	i*i#	T->FB
9	#ABF	i*i#	F->i
10	#ABi	i*i#	
11	#AB	*i#	B->*FB
12	#ABF*	*i#	
13	#ABF	i#	F->i
14	#ABi	i#	
15	#AB	#	B->e
16	#A	#	A->e
17	#	#	Success

结果分析:

1. 非终结符和终结符识别: 程序自动识别出非终结符 (V_n) 为 E T F, 终结符 (V_t) 为 () * + i。
2. 左递归消除: 原文法中 E 和 T 存在直接左递归, 程序简化处理直接左递归 (仅处理 $A \rightarrow A\alpha$ 形式), 消除左递归后的文法为:

- E -> TA
- A -> +TA
- A -> e
- T -> FB
- B -> *FB
- B -> e
- F -> (E)
- F -> i

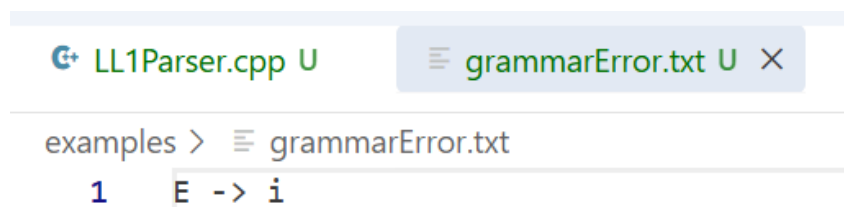
3. FIRST 集计算：计算出各非终结符的 FIRST 集，如 $\text{FIRST}(E) = \{ (i \}$ 、 $\text{FIRST}(T) = \{ (i \}$ 等，这是根据 FIRST 集的计算规则，通过对文法产生式的推导得出的。比如对于 $E \rightarrow TA$ ，因为 T 的 FIRST 集包含 (和 i，所以 E 的 FIRST 集也包含 (和 i。

4. FOLLOW 集计算：计算出各非终结符的 FOLLOW 集，如 $\text{FOLLOW}(E) = \{ \#) \}$ 、 $\text{FOLLOW}(T) = \{ \#) + \}$ 等，其计算依赖于 FIRST 集，通过对文法的分析和特定规则得出。例如，因为 E 是开始符号，所以 # 在 $\text{FOLLOW}(E)$ 中；又因为在 $F \rightarrow (E)$ 中，) 在 E 之后，所以) 也在 $\text{FOLLOW}(E)$ 中。

5. LL (1) 分析表构建：依据 FIRST 集和 FOLLOW 集构建 LL (1) 分析表，表中每个单元格表示在特定非终结符和终结符组合下应用的产生式。例如，在分析表中，当面临非终结符 E 和终结符 (时，应用产生式 $E \rightarrow TA$ ；当面临非终结符 A 和终结符 + 时，应用产生式 $A \rightarrow +TA$ 。

6. 输入串分析：以输入串 $1+2*3$ 为例，程序将其转换为 $i+i*i$ ，然后按照 LL (1) 分析算法进行分析。从初始化栈为 #S（这里 S 即 E），输入串末尾补 # 开始，在分析循环中，根据栈顶元素和当前输入符号查询分析表，不断进行匹配、弹出和压栈操作，直到栈为空且输入串处理完毕，最终得出分析结果为 Success，表示输入串符合该文法规则。

（2）非法输入测试，语法文件 grammarError.txt，输入字符串 (i)
语法文件：



```
LL1Parser.cpp U
examples > grammarError.txt
1 E -> i
```

运行结果：

```
➤ → /workspace git:(master) X ./ll1_parser
请输入文法文件名（例如 grammar.txt）：    examples/grammarError.txt
请输入要分析的输入串： i
```

非终结符(Vn): E
终结符(Vt): i

FIRST集:
FIRST(E)={i}

FOLLOW集:
FOLLOW(E)={#}

LL(1) 分析表 (表格形式):

	#	i
E		E→i

转换后输入串为: i

分析过程:

步骤	分析栈	输入串	所用产生式
1	#E	i#	E→i
2	#i	i#	
3	#	#	Success

结果分析:

1. 栈顶元素 E 面对输入串当前符号 (时, LL (1) 分析表中没有对应的产生式 。
 - 从文法文件可知文法可能较简单 (仅 $E \rightarrow i$) , 只定义了 E 能推出 i , 未涵盖 (等符号的推导规则。输入串 (i) 中 (无法依据现有文法规则处理。
 - 根据此简单文法构建的 LL (1) 分析表, 只有 E 面对 i 时有产生式 $E \rightarrow i$, E 面对 (没有匹配项, 导致分析时遇到错误直接判定失败。

(3) 空情况测试, 语法文件 grammarNull.txt, 输入字符串 # (空串以 # 表示)
语法文件:

```
≡ grammarError.txt U    ≡ grammarNull.txt U X    ≡ ll1
examples > ≡ grammarNull.txt
1    E -> e
```

运行结果:

```
→ /workspace git:(master) X ./l11_parser
请输入文法文件名（例如 grammar.txt）： examples/grammarNull.txt
请输入要分析的输入串： #

非终结符(Vn): E
终结符(Vt): e

FIRST集:
FIRST(E)={e}

FOLLOW集:
FOLLOW(E)={#}

LL(1) 分析表 (表格形式):
+-----+-----+-----+
|         | #       | e       |
+-----+-----+-----+
| E       | E->e   |         |
+-----+-----+-----+
转换后输入串为: #

分析过程:
+-----+-----+-----+-----+
| 步骤   | 分析栈   | 输入串   | 所用产生式 |
+-----+-----+-----+-----+
| 1      | #E       | ##       | E->e       |
+-----+-----+-----+-----+
| 2      | #        | ##       | Success    |
+-----+-----+-----+-----+
```

结果分析:

1. 非终结符和终结符识别: 程序自动识别出非终结符 (Vn) 为 E, 终结符 (Vt) 为 e。这表明程序正确从文法文件中提取了非终结符和终结符, 其中 e 代表空串。
2. 左递归消除: 原文法中未发现左递归 (如 E -> e 形式无左递归), 因此无需进行左递归消除处理。程序直接使用原文法进行后续分析。
3. FIRST 集计算: 计算出非终结符 E 的 FIRST 集为{ e }。这是根据 FIRST 集的计算规则得出的, 由于 E 的唯一产生式是 E -> e, 即 E 可以直接推导出空串, 因此 FIRST (E) 仅包含 e。
4. FOLLOW 集计算: 计算出非终结符 E 的 FOLLOW 集为{ # }。这是因为 E 是开始符号, 根据 FOLLOW 集的计算规则, 开始符号的 FOLLOW 集包含句子结束符#。在这个简单文法中, E 后面没有其他符号, 因此其 FOLLOW 集仅包含#。
5. LL (1) 分析表构建: 依据 FIRST 集和 FOLLOW 集构建 LL (1) 分析表。对于非终结符 E 和终结符 e、#, 分析表中对应的产生式均为 E -> e。这是因为:
当面临终结符 e 时, 由于 FIRST (E) 包含 e, 所以使用产生式 E -> e。
 - 当面临终结符#时, 由于 FIRST (E) 包含 e 且 FOLLOW (E) 包含#, 根据 LL (1) 分析表的构造规则, 同样使用产生式 E -> e。
6. 输入串分析: 输入串为# (表示空串), 程序按照 LL (1) 分析算法进行分析:
 - 初始化: 分析栈为#E, 输入串为## (末尾补#)。
 - 步骤 1: 栈顶元素为 E, 输入符号为#。查询分析表, 应用产生式 E -> e (即弹出 E), 栈变为#。
 - 步骤 2: 栈顶元素为#, 输入符号为#, 二者匹配, 分析成功, 输出 Success。这表明输入串符合该文法规则。

实验结论:

为了验证你第一部分编写的程序是准确的,你设计了什么测试数据?

(1) 样例测试, grammar.txt, 运行结果如下:

● → /workspace git:(master) X g++ -std=c++11 LL1Parser.cpp -o ll1_parser

● → /workspace git:(master) X ./ll1_parser

请输入文法文件名 (例如 grammar.txt): examples/grammar.txt

请输入要分析的输入串: 1+2*3

非终结符(V_n): A B E F T

终结符(V_t): () * + e i

存在左递归, 消除后如下:

A → +TA

A → e

B → *FB

B → e

E → TA

F → (E)

F → i

T → FB

FIRST集:

FIRST(A)={+,e}

FIRST(B)={*,e}

FIRST(E)={(,i}

FIRST(F)={(,i}

FIRST(T)={(,i}

FOLLOW集:

FOLLOW(A)={#,)}

FOLLOW(B)={#,),+}

FOLLOW(E)={#,)}

FOLLOW(F)={#,),*,+}

FOLLOW(T)={#,),+}

LL(1) 分析表 (表格形式):

	#	(	)	*	+	e	i
A	A→e		A→e		A→+TA		
B	B→e		B→e	B→*FB	B→e		
E		E→TA					E→TA
F		F→(E)					F→i
T		T→FB					T→FB

转换后输入串为: i+i*i

分析过程:

步骤	分析栈	输入串	所用产生式
1	#E	i+i*i#	E->TA
2	#AT	i+i*i#	T->FB
3	#ABF	i+i*i#	F->i
4	#ABi	i+i*i#	
5	#AB	+i*i#	B->e
6	#A	+i*i#	A->+TA
7	#AT+	+i*i#	
8	#AT	i*i#	T->FB
9	#ABF	i*i#	F->i
10	#ABi	i*i#	
11	#AB	*i#	B->*FB
12	#ABF*	*i#	
13	#ABF	i#	F->i
14	#ABi	i#	
15	#AB	#	B->e
16	#A	#	A->e
17	#	#	Success

结果分析:

1. 非终结符和终结符识别: 程序自动识别出非终结符 (V_n) 为 E T F, 终结符 (V_t) 为 () * + i。
2. 左递归消除: 原文法中 E 和 T 存在直接左递归, 程序简化处理直接左递归 (仅处理 $A \rightarrow A\alpha$ 形式), 消除左递归后的文法为:

- E -> TA
- A -> +TA
- A -> e
- T -> FB
- B -> *FB
- B -> e
- F -> (E)
- F -> i

3. FIRST 集计算：计算出各非终结符的 FIRST 集，如 $\text{FIRST}(E) = \{ (i \}$ 、 $\text{FIRST}(T) = \{ (i \}$ 等，这是根据 FIRST 集的计算规则，通过对文法产生式的推导得出的。比如对于 $E \rightarrow TA$ ，因为 T 的 FIRST 集包含 (和 i，所以 E 的 FIRST 集也包含 (和 i。

4. FOLLOW 集计算：计算出各非终结符的 FOLLOW 集，如 $\text{FOLLOW}(E) = \{ \#) \}$ 、 $\text{FOLLOW}(T) = \{ \#) + \}$ 等，其计算依赖于 FIRST 集，通过对文法的分析和特定规则得出。

例如，因为 E 是开始符号，所以 # 在 $\text{FOLLOW}(E)$ 中；又因为在 $F \rightarrow (E)$ 中，) 在 E 之后，所以) 也在 $\text{FOLLOW}(E)$ 中。

5. LL (1) 分析表构建：依据 FIRST 集和 FOLLOW 集构建 LL (1) 分析表，表中每个单元格表示在特定非终结符和终结符组合下应用的产生式。例如，在分析表中，当面临非终结符 E 和终结符 (时，应用产生式 $E \rightarrow TA$ ；当面临非终结符 A 和终结符 + 时，应用产生式 $A \rightarrow +TA$ 。

6. 输入串分析：以输入串 1+2*3 为例，程序将其转换为 $i+i*i$ ，然后按照 LL (1) 分析算法进行分析。从初始化栈为 #S（这里 S 即 E），输入串末尾补 # 开始，在分析循环中，根据栈顶元素和当前输入符号查询分析表，不断进行匹配、弹出和压栈操作，直到栈为空且输入串处理完毕，最终得出分析结果为 Success，表示输入串符合该文法规则。

(2) 非法输入测试，语法文件 grammarError.txt，输入字符串 (i)
语法文件：

```
LL1Parser.cpp U  grammarError.txt U X
examples > grammarError.txt
1 E -> i
```

运行结果：

```
→ /workspace git:(master) X ./ll1_parser
请输入文法文件名（例如 grammar.txt）： examples/grammarError.txt
请输入要分析的输入串： i

非终结符(Vn): E
终结符(Vt): i

FIRST集:
FIRST(E)={i}

FOLLOW集:
FOLLOW(E)={#}

LL(1) 分析表（表格形式）:
+-----+-----+-----+
|         | #     | i     |
+-----+-----+-----+
| E       |       | E->i  |
+-----+-----+-----+
转换后输入串为: i

分析过程:
+-----+-----+-----+-----+
| 步骤   | 分析栈   | 输入串   | 所用产生式 |
+-----+-----+-----+-----+
| 1      | #E       | i#       | E->i       |
+-----+-----+-----+-----+
| 2      | #i       | i#       |            |
+-----+-----+-----+-----+
| 3      | #        | #        | Success    |
+-----+-----+-----+-----+
```

结果分析：

栈顶元素 E 面对输入串当前符号 (时，LL (1) 分析表中没有对应的产生式 。

- 从文法文件可知文法可能较简单（仅 $E \rightarrow i$ ），只定义了 E 能推出 i，未涵盖 (等符号的推导规则。输入串 (i) 中 (无法依据现有文法规则处理。
- 根据此简单文法构建的 LL (1) 分析表，只有 E 面对 i 时有产生式 $E \rightarrow i$ ，E 面对 (没有匹配项，导致分析时遇到错误直接错误。

(3) 空情况测试，语法文件 grammarNull.txt，输入字符串 #（空串以 # 表示）

语法文件：

```
≡ grammarError.txt U  ≡ grammarNull.txt U X  ≡ ll1
examples > ≡ grammarNull.txt
1  E -> e
```

结果分析：

```
→ /workspace git:(master) X ./ll1_parser
请输入文法文件名（例如 grammar.txt）： examples/grammarNull.txt
请输入要分析的输入串： #

非终结符 (Vn)： E
终结符 (Vt)： e

FIRST集：
FIRST(E)={e}

FOLLOW集：
FOLLOW(E)={#}

LL(1) 分析表（表格形式）：
+-----+-----+
|      | #      | e      |
+-----+-----+
| E     | E->e   |        |
+-----+-----+

转换后输入串为： #

分析过程：
+-----+-----+-----+-----+
| 步骤   | 分析栈   | 输入串   | 所用产生式   |
+-----+-----+-----+-----+
| 1      | #E       | ##       | E->e         |
+-----+-----+-----+-----+
| 2      | #        | ##       | Success      |
+-----+-----+-----+-----+
```

1. 非终结符和终结符识别：程序自动识别出非终结符 (Vn) 为 E，终结符 (Vt) 为 e。这表明程序正确从文法文件中提取了非终结符和终结符，其中 e 代表空串。
2. 左递归消除：原文法中未发现左递归（如 $E \rightarrow e$ 形式无左递归），因此无需进行左递归消除处理。程序直接使用原文法进行后续分析。
3. FIRST 集计算：计算出非终结符 E 的 FIRST 集为{ e }。这是根据 FIRST 集的计算规则得出的，由于 E 的唯一产生式是 $E \rightarrow e$ ，即 E 可以直接推导出空串，因此 FIRST (E) 仅包含 e。
4. FOLLOW 集计算：计算出非终结符 E 的 FOLLOW 集为{ # }。这是因为 E 是开始符号，根据 FOLLOW 集的计算规则，开始符号的 FOLLOW 集包含句子结束符#。在这个简单文法中，E 后面没有其他符号，因此其 FOLLOW 集仅包含#。
5. LL (1) 分析表构建：依据 FIRST 集和 FOLLOW 集构建 LL (1) 分析表。对于非终结符 E 和终结符 e、#，分析表中对应的产生式均为 $E \rightarrow e$ 。这是因为：当面临终结符 e 时，由于 FIRST (E) 包含 e，所以使用产生式 $E \rightarrow e$ 。

- 当面临终结符#时, 由于 FIRST (E) 包含 e 且 FOLLOW (E) 包含#, 根据 LL (1) 分析表的构造规则, 同样使用产生式 $E \rightarrow e$ 。
6. 输入串分析: 输入串为# (表示空串), 程序按照 LL (1) 分析算法进行分析:
- 初始化: 分析栈为#E, 输入串为## (末尾补#)。
 - 步骤 1: 栈顶元素为 E, 输入符号为#。查询分析表, 应用产生式 $E \rightarrow e$ (即弹出 E), 栈变为#。
 - 步骤 2: 栈顶元素为#, 输入符号为#, 二者匹配, 分析成功, 输出 Success。这表明输入串符合该文法规则。

心得体会:

通过本次实验，我完整实现了从文法输入、左递归消除、FIRST/FOLLOW 集计算、LL (1) 分析表构造到预测分析全过程的程序设计，不仅加深了对编译原理理论知识的理解，也提升了将算法转化为代码的实践能力。

在实验过程中,我深刻体会到理论与实践结合的重要性。课堂上学习的 FIRST 集、FOLLOW 集、LL (1) 文法条件等概念,在编码时才真正理解其作用: FIRST 集决定产生式的选择入口, FOLLOW 集处理空产生式的匹配场景,二者共同保证 LL (1) 分析表无冲突。调试时遇到的 FIRST 集全空、FOLLOW 集缺少结束符#、文件路径错误等问题,让我明白迭代收敛、初始化规范、边界处理是程序正确运行的关键。

同时，我对自顶向下语法分析有了更直观的认识：它从开始符号出发，试图通过产生式推导出输入串，依靠分析表与栈结构实现高效无回溯匹配，结构清晰、易于编程。但它也要求文法必须满足 LL(1) 条件，必须消除左递归与提取左公因子，这让我理解了文法预处理的必要性。

本次实验让我系统掌握了 LL (1) 预测分析器的完整实现流程，提升了调试能力与逻辑思维，也为后续学习语法分析、中间代码生成等编译原理内容打下了坚实基础。

指导教师批阅意见:

成绩评定:

指导教师签字：
年 月 日

备注: